Unreal Engine MCP Python Bridge Plugin

This is a plugin for Unreal Engine (UE) that creates a server implementation of Model Context Protocol (MCP). This allows MCP clients, like Anthropic's Claude, to access the full UE Python API.

Use Cases

- Developing tools and workflows in Python with an agent like Claude.
- Intelligent and dynamic automation of such workflows.
- General collaborative development with an agent.

Prerequisites

- Visual Studio 2019 or higher.
- An AI Agent. Below, we assume Claude will be used. But any AI Agent that implements MCP should suffice.
- Unreal Engine 5 with the Python Editor Script Plugin enabled.
- Note the Unreal Engine Python API.

Installing from GitHub

1. Create a new Unreal Engine C++ project.
2. Under the project root directory, find the `Plugins` folder.
3. Clone the UnrealMCPBridge repo into the `Plugins` folder so that there is a new containing folder with all the project contents underneath.
4. Right-click your UE project file (ends with `.uproject`) and select "Generate Visual Studio project files". If you don't immediately see that option, first select "Show more options" and it should appear.
5. Open your new Visual Studio project and build.

Installing from Fab

1. After purchasing the plugin on Fab, find the plugin in your Library in the Epic Games Launcher.
2. Click "Install to Engine" and choose the appropriate version of Unreal Engine.

Configure Claude for Desktop

1. Copy `unreal_mcp_client.py` from the MCPClient folder on GitHub to a location of your choice.

2. Find your `claude_desktop_config.json` configuration file. Mac location: `~/Library/Application\ Support/Claude/claude_desktop_config.json` Windows location: `[path_to_your_user_account]\AppData\Roaming\Claude\claude_desktop_config.json`

3. Add the `unreal-engine` server section to your config file and update the path location excluding the square brackets, below.

    - Mac path format: `/[path_from_step_1]/unreal_mcp_client.py`
    - Windows path format: `C:\\[path_from_step_1]\\unreal_mcp_client.py`

```
{
  "mcpServers": {
    "unreal-engine": {
      "command": "python",
      "args": [
        "[mac_or_windows_format_path_to_unreal_mcp_client.py]"
      ]
    }
  }
}
```

4. Create a new Unreal Engine project or load an existing project.
5. From the "Edit" menu, select "Plugins".
6. Enable these plugins: `UnrealMCPBridge` and `Python Editor Script Plugin`.
7. Restart Unreal Engine.
8. There should be a new toolbar button that says, "Start MCP Bridge" when you hover with your mouse.
9. Click the MCP Bridge button. A pop-up will state, "MCP Python bridge running." The Output Log will note a new socket server listening on 127.0.0.1:9000.
10. Launch Claude as an administrator.
11. Click the "Attach from MCP" plug-icon. Under "Choose an integration" are two test Prompts: `create_castle` and `create_town`. You can edit their implementations in `unreal_server_init.py` under the `Content` folder of the plugin. Be sure to restart Unreal Engine after any changes.
12. Alternatively, ask Claude to build in your project using the UE Python API.
13. A list of currently implemented tools can be found by clicking the hammer-icon to the left of the plug-icon.

Configure Claude Code

Claude Code is Anthropic's CLI tool for agentic coding. It supports MCP servers via a `.mcp.json` configuration file in your project root.

Option A: CLI Command (Recommended)

From your Unreal Engine project root directory, run:

Mac/Linux:

```
claude mcp add --transport stdio --scope project unreal-bridge -- python /path/to/unreal_mcp_client.py
```

Windows:

```
claude mcp add --transport stdio --scope project unreal-bridge -- python C:\path\to\unreal_mcp_client.py
```

This creates a `.mcp.json` file in your project root automatically.

Option B: Manual Configuration

1. Copy `unreal_mcp_client.py` from the MCPClient folder to a location of your choice.

2. Create a `.mcp.json` file in your Unreal Engine project root directory (the folder containing your `.uproject` file) with the following contents:

Mac/Linux:

```
{
  "mcpServers": {
    "unreal-bridge": {
      "command": "python",
      "args": ["/path/to/unreal_mcp_client.py"],
      "env": {}
    }
  }
}
```

Windows (use forward slashes or escaped backslashes):

```
{
  "mcpServers": {
    "unreal-bridge": {
      "command": "python",
      "args": ["C:/path/to/unreal_mcp_client.py"],
      "env": {}
    }
  }
}
```

Starting a Session

1. Open your Unreal Engine project and click the MCP Bridge toolbar button to start the server.
2. Open a terminal in your project root directory and launch Claude Code:

```
claude
```

3. Claude Code will detect the `.mcp.json` and prompt you to approve the MCP server on first use.
4. Once approved, all MCP tools (get_actors, spawn_actor, execute_python, etc.) are available. Claude Code can call them directly during the conversation.

Verifying the Connection

Inside a Claude Code session, type `/mcp` to check the status of connected MCP servers. You should see the `unreal-bridge` server listed as connected.

Available Tools

These tools are defined in `unreal_mcp_client.py` and are available to the AI agent once the MCP bridge is connected. In Claude Desktop they appear under the hammer-icon. In Claude Code they can be called directly during conversation.

Project & Asset Discovery

Tool	Parameters	Description
`get_project_dir`	—	Returns the top-level project directory path (e.g., `D:/MyProject/`).

Tool	Parameters	Description
`get_content_dir`	—	Returns the Content directory path (e.g., `D:/MyProject/Content/`).
`find_basic_shapes`	—	Returns a list of built-in basic shapes (cube, sphere, cylinder, etc.) that can be used for building.
`find_assets`	`asset_name`	Searches the asset registry for assets matching a name. Use keywords like `Floor`, `Wall`, `Door` to discover meshes.
`get_asset`	`asset_path`	Returns the bounding box dimensions of a static mesh asset.

Actor Management

Tool	Parameters	Description
`get_actors`	—	Lists all actors in the current level with their name, class, and location.
`get_actor_details`	`actor_name`	Returns all properties and details for a specific actor by name.
`get_selected_actors`	—	Returns the actors currently selected in the editor viewport. Useful for inspecting what the user is looking at.
`spawn_actor`	`asset_path`, `location_x/y/z`, `rotation_x/y/z`, `scale_x/y/z`	Spawns a static mesh actor at the given location, rotation, and scale. All position/rotation/scale parameters default to 0.

Tool	Parameters	Description
`modify_actor`	`actor_name`, `property_name`, `property_value`	Modifies a property of an existing actor. The value is a string that gets converted to the appropriate type on the server side.
`set_material`	`actor_name`, `material_path`	Applies a material to a static mesh actor. Use asset paths like `/Game/Materials/M_MyMaterial`.
`delete_all_static_mesh_actors`	—	Deletes **all** static mesh actors in the scene. Use with caution — this removes everything, not just what was recently placed.

Building Utilities

Tool	Parameters	Description
`create_grid`	`asset_path`, `grid_width`, `grid_length`	Creates a grid of tiles using the given asset, automatically spaced based on the asset's bounding box. `grid_width` and `grid_length` are the number of tiles in each dimension.
`create_town`	`town_center_x/y`, `town_width`, `town_height`	Creates a procedural town layout centered at the given position using available assets.

Blueprint Execution

Tool	Parameters	Description
`run_blueprint_function`	`blueprint_name`, `function_name`, `arguments`	Executes a function defined in a Blueprint. Arguments are passed as a comma-separated string.

Blueprint Graph Authoring (Reads)

These tools introspect a Blueprint's graphs without `execute_python`. Together they let an agent walk an event graph or function graph node-by-node, follow exec/data pin connections, and read every kind of node reference. `blueprint_path` is an asset path like `/Game/BP/BP_NPC.BP_NPC` (or just `/Game/BP/BP_NPC` — both resolve). `node_id` strings come from a list call (`get_node_ids`); IDs may shift after compile, so re-introspect rather than caching them.

Tool	Parameters	Description
`get_node_ids`	`blueprint_path`	Returns the node IDs in the BP's event graph (JSON list).
`get_node_ids_in_graph`	`blueprint_path`, `graph_name`	Returns node IDs in a named graph (event graph, user function, or macro).
`get_pin_names`	`blueprint_path`, `node_id`	Returns the internal pin names on a node ("execute", "then", "ReturnValue", etc.).
`get_pin_details`	`blueprint_path`, `node_id`	Returns pin info as `"PinName|Direction|Type"` strings (Input/Output, exec/bool/int/struct/object/...).
`get_node_title`	`blueprint_path`, `node_id`	Returns the human-readable list-view title (e.g., "Print String", "Branch", "Get MyVar").
`get_function_reference`	`blueprint_path`, `node_id`	For a `K2Node_CallFunction`,

Tool	Parameters	Description
		returns `"MemberParent::MemberName"` (e.g., `"KismetSystemLibrary::PrintString"`). Empty for non-call nodes.
`get_event_reference`	`blueprint_path`, `node_id`	For `K2Node_Event` returns `"Class::Member"`; for `K2Node_CustomEvent` returns the custom event name.
`get_variable_reference`	`blueprint_path`, `node_id`	For `K2Node_VariableGet` / `K2Node_VariableSet`, returns the variable name.
`get_macro_reference`	`blueprint_path`, `node_id`	For `K2Node_MacroInstance`, returns `"Library::MacroName"`.
`get_pin_connections`	`blueprint_path`, `node_id`, `pin_name`	Returns the linked-to pins as `"OtherNodeId.OtherPinName"` strings (JSON list). The key function for walking a graph: follow exec pins for control flow, follow data pins for value flow. Silent no-op if the pin doesn't exist on the node.
`get_pin_default_value`	`blueprint_path`, `node_id`, `pin_name`	Returns the default literal of an unconnected input pin. For object/class pins returns the asset path; for value pins returns the literal string.
`get_function_graph_names`	`blueprint_path`	Lists user-defined function graphs on a BP. Pair with `get_node_ids_in_graph` to walk them.

Tool	Parameters	Description
`get_macro_graph_names`	`blueprint_path`	Lists macro graphs on a BP.
`get_member_variable_names`	`blueprint_path`	Lists member variables (`NewVariables` — distinct from components and from variable nodes that reference them).
`get_member_variable_type`	`blueprint_path`, `variable_name`	Returns a richer type string: primitives as-is (`int`, `double`), or `struct:Name`, `object:/Path`, `class:/Path`, `enum:Name`, `TArray<...>`, `TSet<...>`, `TMap<K,V>`.
`get_function_metadata`	`blueprint_path`, `function_graph_name`	Returns `"Category|Access|Pure|Const"`. Access ∈ {Public, Protected, Private}.
`get_function_parameters`	`blueprint_path`, `function_graph_name`	Returns input/output parameters as `"Name|Direction|Type|DefaultValue"` strings. Direction ∈ {Input, Output}. DefaultValue is empty for outputs.
`get_overridable_functions`	`blueprint_path`	Lists parent-class `BlueprintEvent` functions that aren't yet overridden — i.e., what events the BP could implement but doesn't.
`get_component_template_names`	`blueprint_path`	Lists Simple Construction Script (SCS) component variable names.
`get_component_template_class`	`blueprint_path`, `component_name`	Returns the class name of an SCS component (e.g., `"StaticMeshComponent"`).

Blueprint Graph Authoring (Writes)

These tools build BP graphs without `execute_python`. The typical authoring sequence is: create BP → add events/calls/variables → wire pins → set defaults → compile and save. Node IDs returned by `add_*_node` tools are stable until the next compile.

Tool	Parameters	Description
`create_new_blueprint`	`path`, `name`, `parent_class_path`	Creates a new BP asset. Returns its full asset path (e.g., `/Game/BP/BP_MyActor.BP_MyActor`). New BPs are created with default `BeginPlay` / `Tick` / `ActorBeginOverlap` events already present — find them via `get_node_ids` rather than adding duplicates.
`add_event_node`	`blueprint_path`, `event_name`, `pos_x`, `pos_y`	Adds an event node (e.g., `"ReceiveDestroyed"`). Avoid duplicating BeginPlay/Tick — they already exist on new BPs.
`add_call_function_node`	`blueprint_path`, `target_class_path`, `function_name`, `pos_x`, `pos_y`	Adds a function call. `target_class_path` examples: `/Script/Engine.KismetSystemLibrary`, `/Script/Engine.KismetMathLibrary`, or any BP class path.
`add_spawn_actor_node`	`blueprint_path`, `actor_class_path`, `pos_x`, `pos_y`	Adds the canonical `UK2Node_SpawnActorFromClass`. The class is pinned and `ExposeOnSpawn` properties materialize as input pins automatically. Also auto-spawns a `Make Transform` helper node

Tool	Parameters	Description
		(KismetMathLibrary call) wired into `SpawnTransform`, because that pin is by-ref and the K2 compiler rejects an unconnected one. Edit the Make Transform's Location/Rotation/Scale sub-pins to set a non-identity spawn transform, or replace it with your own input.
`add_variable_get_node`	`blueprint_path`, `variable_name`, `pos_x`, `pos_y`	Adds a variable Get node. Variable must already exist (via `add_variable`).
`add_variable_set_node`	`blueprint_path`, `variable_name`, `pos_x`, `pos_y`	Adds a variable Set node. Variable must already exist.
`add_branch_node`	`blueprint_path`, `pos_x`, `pos_y`	Adds an if/else `Branch` node.
`add_custom_event_node`	`blueprint_path`, `event_name`, `pos_x`, `pos_y`	Adds a Custom Event entry point callable from BP code.
`connect_pins`	`blueprint_path`, `source_node_id`, `source_pin`, `target_node_id`, `target_pin`	Wires two pins (exec or data). Use the internal pin names (`"then"`, `"execute"`, `"ReturnValue"`, etc.) — call `get_pin_names` first if unsure.
`set_pin_default_value`	`blueprint_path`, `node_id`, `pin_name`, `value`	Sets a literal default on an unconnected input. For object/class pins pass an asset path (`"/Game/BP/BP_NPC.BP_NPC_C"`).

Tool	Parameters	Description
`add_variable`	`blueprint_path`, `name`, `type_name`, `instance_editable`	Adds a member variable. Type names: `bool`, `int`, `float`, `string`, `name`, `text`, `Vector`, `Rotator`, `Transform`, or a class path.
`add_function_graph`	`blueprint_path`, `graph_name`	Creates a new user-defined function graph on the BP (empty FunctionEntry, no parameters yet).
`add_function_parameter`	`blueprint_path`, `function_graph_name`, `param_name`, `type_name`, `is_input`, `default_value`	Adds an input or output parameter to a function graph. If adding the first output, a `FunctionResult` node is auto-spawned.
`compile_and_save_blueprint`	`blueprint_path`	Compiles the BP and writes to disk. Returns `"True"` on success without errors. Call once at the end of an authoring sequence.

PIE Control

Drive Play-In-Editor sessions programmatically — no toolbar clicks. Useful for net trace capture, automation tests, AI/replication smoke tests.

Tool	Parameters	Description
`start_pie`	`net_mode`, `num_clients`	Starts PIE. `net_mode` ∈ `{"standalone", "listen_server", "client"}`. `num_clients` ≥ 1. Sets `ULevelEditorPlaySettings::PlayNetMode` + `PlayNumberOfClients`, then calls `LevelEditorSubsystem::EditorRequestBeginPlay`

Tool	Parameters	Description
		. PIE start is async — verify with `is_pie_running`.
`stop_pie`	—	Stops the active PIE session.
`is_pie_running`	—	Returns `"True"` if a PIE world is currently active.

Viewport & Screenshots

Tool	Parameters	Description
`take_screenshot`	—	Captures the active editor viewport to a PNG file and returns the file path.
`set_viewport_camera`	`location_x/y/z`, `rotation_pitch/yaw/roll`	Moves the editor viewport camera to a specific location and rotation. Pitch = look up/down, Yaw = compass heading, Roll = tilt.
`focus_viewport_on_actor`	`actor_name`, `distance`	Points the camera at a named actor from a 45-degree overhead angle. Distance is auto-calculated from the actor's bounds, or can be overridden (set to 0 for auto).

CSV Profiler (High-Level Performance)

The CSV profiler captures per-frame aggregate stats — frame time, thread times, GPU time, draw calls, memory — without the overhead of a full Insights trace. Use it as a quick health check to identify *which area* has a problem, then follow up with Insights tracing to find the *specific cause*.

> **UE 5.5 limitation:** do not run the CSV profiler and an Insights trace concurrently — together they trip an engine assertion (`CsvProfilerProvider.cpp:170`, `CurrentCapture` failed) and crash the editor. UE 5.6 and 5.7 are unaffected. On 5.5 use them sequentially: capture one, stop, then capture the other.

Tool	Parameters	Description
`start_csv_profile`	—	Starts the CSV profiler. Captures per-frame stats for every subsequent frame.
`stop_csv_profile`	—	Stops the CSV profiler. The output CSV file needs a few seconds to finalize before reading.
`get_csv_profile`	—	Reads the most recent CSV profile and returns a parsed JSON summary with timing stats, counters, budget analysis, and trend data.

Call order and timing:

```
1. start_csv_profile       → profiler begins capturing
2. [perform actions]       → play in editor, stress test, etc.
3. stop_csv_profile        → sends the stop command (returns immediately)
4. [wait 2-3 seconds]      → the CSV profiler writes asynchronously on the
game thread; it needs a few engine ticks to flush the file to disk
5. get_csv_profile         → reads and parses the CSV, returns the summary
```

The wait between `stop_csv_profile` and `get_csv_profile` is important. The `csvprofile stop` console command is asynchronous — the engine needs game thread ticks to finalize and close the CSV file. If `get_csv_profile` is called too soon, the file may still be locked. If that happens, simply wait a moment and call `get_csv_profile` again.

What the summary includes:

Section	Contents
timing	avg/min/max/p50/p95/p99 for FrameTime, GameThreadTime, RenderThreadTime, GPUTime, RHIThreadTime, InputLatencyTime

Section	Contents
counters	avg/min/max/p50/p95/p99 for DrawCalls, PrimitivesDrawn, MemoryFreeMB, PhysicalUsedMB
budget	Total frames, frames over 60fps/30fps budgets (count + percentage), average FPS
trend	First-half vs second-half average frame time, percent change, and label (stable/degrading/improving)

Interpreting the results: Whichever thread time is closest to FrameTime is the bottleneck. If GameThreadTime dominates, the scene is CPU game-bound (too many actors ticking, expensive Blueprint logic). If RenderThreadTime dominates, it's CPU render-bound (too many draw calls, complex scene setup). If GPUTime dominates, it's GPU-bound (heavy shaders, too many triangles, post-processing). The trend section reveals whether performance is steady or degrading over time (e.g., from a memory leak or growing actor count).

Unreal Insights Tracing (Function-Level Detail)

Insights tracing captures the full call stack — individual function names, source file/line numbers, per-scope timing with nesting. Use it after the CSV profiler identifies which area is the problem, to pinpoint the exact function or Blueprint causing it.

Tool	Parameters	Description
`start_trace`	—	Starts capturing an Unreal Insights trace to file. Captures CPU, GPU, Frame, Counters, Region, and **Net** channels. Also issues `NetTrace.SetTraceVerbosity 2` so net events actually emit (the channel alone isn't enough — runtime verbosity must be non-zero). Call `stop_trace` when done.

Tool	Parameters	Description
`stop_trace`	—	Stops the current trace capture. Returns the absolute path to the `.utrace` file.
`analyze_trace`	`trace_path`, `top_n`	Parses a `.utrace` file and returns a JSON summary including: trace duration, thread info, frame statistics (avg/min/max FPS), and the top N most expensive timing scopes sorted by total inclusive time. Each scope includes name, source file/line, call count, and total/avg/max inclusive time in milliseconds. Default `top_n` = 50.
`get_trace_spikes`	`trace_path`, `budget_ms`, `max_frames`, `top_scopes_per_frame`	Finds frames that exceeded a time budget. For each spike frame, returns the top timing scopes contributing to that frame. Default budget = 33.33ms (30fps).
`get_trace_frame_summary`	`trace_path`	Returns frame timing statistics from a `.utrace` file: frame count, avg/min/max frame time, FPS, percentiles (p50/p90/p95/p99), and counts of frames exceeding 60fps and 30fps budgets.
`get_trace_net_summary`	`trace_path`, `top_n`	Parses the Net channel: per-connection packet/byte totals (in/out + drop rate), top objects by outbound bandwidth (which actors send the most data), top event types by outbound bandwidth

Tool	Parameters	Description
		(which RPCs / replicated properties dominate), and game-instance metadata (server vs client, Iris on/off). Reports `has_net_data: false` cleanly for non-networked traces.

> **UE 5.5 limitation:** see the CSV Profiler note above — don't run a trace and CSV profile concurrently on UE 5.5 (engine assertion crash). Use them sequentially. UE 5.6 and 5.7 are fine.

Python Execution

Tool	Parameters	Description
`execute_python`	`code`	Executes arbitrary Python code inside the Unreal Engine process. This gives the agent access to the full Unreal Engine Python API. Returns printed output and error tracebacks.

`execute_python` is the most powerful tool. The agent can import `unreal`, call any editor subsystem, manipulate assets, modify materials, create Blueprints, and run any logic that the UE Python API supports. The other tools are convenience wrappers for common operations that don't require writing Python code.

Available Prompts

Prompts are pre-written instructions that guide the agent through a multi-step workflow. In Claude Desktop, they appear under the plug-icon.

Prompt	Description
`create_castle`	Clears the scene, finds basic shapes, and builds a castle from primitives.

Prompt	Description
`create_town`	Clears the scene, finds a floor tile, builds a grid, and generates a town layout.

You can add your own prompts by following the pattern in the "Developing New Tools and Prompts" section below.

C++ Libraries

The plugin bundles five C++ `BlueprintFunctionLibrary` classes that auto-reflect to Python when the plugin is loaded. Most of `BlueprintGraphLibrary`, `TraceAnalysisLibrary`, and `PIEControlLibrary` is now exposed as dedicated MCP tools (see the tool tables above). The `execute_python` tool can also reach them directly via `import unreal`.

Library	Python Access	Purpose
`BlueprintGraphLibrary`	`unreal.BlueprintGraphLibrary`	Read and write Blueprint graphs: create BPs, add event/function/variable/branch/spawn-actor/macro nodes, wire pins, set defaults, author user function graphs with typed parameters, compile and save. Also full introspection — node titles, function/event/variable references, pin connections and defaults, function metadata, override-able parent functions, member variables, SCS components. **Most operations are exposed as dedicated MCP tools** under "Blueprint Graph Authoring".
`NiagaraEditorLibrary`	`unreal.NiagaraEditorLibrary`	Create Niagara particle systems, add emitters and modules, set parameters, configure renderers (Sprite/Ribbon/Light/Mesh/De

Library	Python Access	Purpose
		cal), compile. Used via `execute_python`.
`ViewportCaptureLibrary`	`unreal.ViewportCaptureLibrary`	Synchronous viewport screenshot capture via ReadPixels. Used internally by `take_screenshot`.
`TraceAnalysisLibrary`	`unreal.TraceAnalysisLibrary`	Parse Unreal Insights `.utrace` files — timing scopes, frame stats, thread info, **and Net channel events** (per-connection bytes/packets, top objects/event types by replication bandwidth) via `INetProfilerProvider`. Used internally by `analyze_trace`, `get_trace_spikes`, `get_trace_frame_summary`, `get_trace_net_summary`.
`PIEControlLibrary`	`unreal.PIEControlLibrary`	Start, stop, and query Play-In-Editor sessions with optional multiplayer config. Wraps `ULevelEditorPlaySettings` + `LevelEditorSubsystem`. Exposed as `start_pie` / `stop_pie` / `is_pie_running` MCP tools.

Developing New Tools and Prompts

Examine `unreal_mcp_client.py` and you'll see how MCP defines tools and prompts using Python decorators above functions. As an example:

```python
@mcp.tool()
def get_project_dir() -> str:
    """Get the top level project directory"""
    result = send_command("get_project_dir")
    if result.get("status") == "success":
        response = result.get("result")
        return response
    else:
        return json.dumps(result)
```

This sends the `get_project_dir` command to Unreal Engine for execution and returns the project level directory for the current project. Under the `Content` folder of the plugin, you will see the server-side implementation of this tool command:

```python
@staticmethod
def get_project_dir():
    """Get the top level project directory"""
    try:
        project_dir = unreal.Paths.project_dir()
        return json.dumps({
            "status": "success",
            "result" : f"{project_dir}"
        })
    except Exception as e:
        return json.dumps({ "status": "error", "message": str(e) })
```

Follow this pattern to create new tools that appear in the Claude Desktop interface under the hammer-icon, or as callable tools in Claude Code.

Implementing new prompts is a matter of adding them to `unreal_mcp_client.py`. As an example, here is the `create_castle` prompt from `unreal_mcp_client.py`:

```python
@mcp.prompt()
def create_castle() -> str:
    """Create a castle"""
    return f"""
Please create a castle in the current Unreal Engine project.
0. Refer to the Unreal Engine Python API when creating new python code:
https://dev.epicgames.com/documentation/en-us/unreal-engine/python-api/?app
lication_version=5.7
1. Clear all the StaticMeshActors in the scene.
```

```
2. Get the project directory and the content directory.
3. Find basic shapes to use for building structures.
4. Create a castle using these basic shapes.
"""
```

Be sure to maintain triple-quotes so the entire prompt is returned. A good way to iterate over creating prompts is simply iterating each step number with Claude until you get satisfactory results. Then combine them all into a numbered step-by-step prompt as shown.

You must restart Claude for any changes to `unreal_mcp_client.py` to take effect. Note for Windows, you might need to end the Claude process in the Task Manager to truly restart Claude. For Claude Code, restart the session or use `/mcp` to reconnect.

Creating Blueprints and Niagara Systems

The `execute_python` tool gives the agent access to the full Unreal Engine Python API, including the C++ wrapper libraries bundled with this plugin (see table above).

Creating and Editing Blueprints

Ask the agent to author or modify a Blueprint and it can do everything end-to-end with the dedicated MCP tools (see "Blueprint Graph Authoring" tables above) — no `execute_python` needed for graph work. The typical authoring sequence is:

1. `create_new_blueprint` to create the asset
2. `add_variable` for any member variables
3. `get_node_ids` + `get_event_reference` to find existing default events (BeginPlay/Tick/ActorBeginOverlap are auto-created on new BPs)
4. `add_call_function_node` / `add_spawn_actor_node` / `add_branch_node` / etc. to add new nodes
5. `connect_pins` to wire exec and data pins
6. `set_pin_default_value` to set literal inputs
7. `compile_and_save_blueprint`
8. Read it back via `get_node_ids` + `get_node_title` + `get_function_reference` + `get_pin_connections` to verify.

For introspecting *existing* Blueprints — listing function graphs, reading parameter signatures, finding which events are overridden, walking exec/data flow — use the read tools (`get_function_graph_names`, `get_function_parameters`, `get_overridable_functions`, `get_pin_connections`, etc.). Useful before modifying a BP so the agent doesn't duplicate or clobber existing logic.

Example prompt for Claude Code — type directly in the conversation:

```
Create a Blueprint at /Game/Blueprints/BP_DayNightCycle that has a public
float variable called "CycleSpeed" and a DirectionalLight reference
variable called "SunLight". On BeginPlay it should print "Day/Night cycle
started" to the screen.
```

Example: defining a typed function on a Blueprint

```
On /Game/BP/BP_NPC, add a user function "ComputeDamage" with inputs
(BaseDamage: float = 10.0, ArmorClass: int) and output (Result: float).
Then compile and save.
```

This translates to `add_function_graph("/Game/BP/BP_NPC", "ComputeDamage")` followed by three `add_function_parameter` calls and a `compile_and_save_blueprint`. No `execute_python` involved.

Example: scripting multiplayer PIE for net trace capture

```
Start a trace, launch a 2-client listen-server PIE session for 5 seconds,
stop, and analyze the net channel.
```

The agent calls `start_trace`, `start_pie("listen_server", 2)`, waits, `stop_pie`, `stop_trace`, then `get_trace_net_summary` on the returned `.utrace` path — surfacing per-connection bandwidth, top replicated actors, and dominant RPCs/properties.

Creating Niagara Particle Effects

Ask the agent to create a Niagara system and it will use `execute_python` to call `NiagaraEditorLibrary` functions. The library supports creating systems from scratch, adding emitters with modules (spawn rate, forces, velocity, etc.), configuring renderers (Sprite, Ribbon, Light, Mesh, Decal), and setting parameters.

Example prompt for Claude Desktop — add this to `unreal_mcp_client.py`:

```
@mcp.prompt()
def create_rain() -> str:
    """Create a rain particle effect"""
    return f"""
Create a rain Niagara particle system using the NiagaraEditorLibrary.
```

```
Use execute_python with `import unreal` and `unreal.NiagaraEditorLibrary`
to:
1. Create a new Niagara system at /Game/VFX/NS_Rain.
2. Add an empty emitter named "RainDrops".
3. Add these modules in order:
   - SpawnRate (EmitterUpdate) -- set to 500 particles/sec.
   - BoxLocation (ParticleSpawn) -- set Box Size to (3000, 3000, 50).
   - AddVelocity (ParticleSpawn) -- set Velocity to (200, 0, -1500) for
angled rain.
   - GravityForce (ParticleUpdate).
   - SolveForcesAndVelocity (ParticleUpdate).
4. Set Initialize Particle lifetime to 2.0 seconds.
5. Set Initialize Particle sprite size to (1, 15) for thin streaks.
6. Set the sprite renderer to VelocityAligned facing.
7. Compile and save the system.
8. Spawn the system into the level at (0, 0, 1500).
"""
```

Example prompt for Claude Code — type directly in the conversation:

```
Create a Niagara lightning effect with a beam emitter. It should be a
jagged blue-white bolt that fires once from (0, 0, 2000) down to (0, 0, 0).
Use a Ribbon renderer with JitterPosition for the jagged shape.
```

Analyzing Performance

The plugin provides two levels of performance analysis that work together:

1. **CSV Profiler** — Quick, high-level dashboard. Answers: "How is overall performance? Which thread is the bottleneck?"
2. **Unreal Insights** — Deep, function-level detail. Answers: "Which specific function or Blueprint is causing the bottleneck?"

Recommended workflow — triage then drill down:

```
CSV Profiler (high-level triage):
  1. start_csv_profile          → begins per-frame stat capture
  2. [perform actions]          → play in editor, stress test
  3. stop_csv_profile           → sends stop command
  4. [wait 2-3 seconds]         → CSV file needs game thread ticks to
flush
```

```
  5. get_csv_profile            → parsed summary: timing, counters,
budget, trend

Unreal Insights (deep dive, when CSV profiler flags a problem):
  6. start_trace                → begins detailed .utrace capture
  7. [reproduce the problem]    → same scenario that CSV profiler flagged
  8. stop_trace                 → returns .utrace file path
  9. analyze_trace(path, 20)    → top 20 most expensive functions with
call counts and durations
 10. get_trace_spikes(path)     → frames that exceeded budget, with
per-frame hotspot breakdown
 11. get_trace_frame_summary    → frame time percentiles
(p50/p90/p95/p99), FPS stats
```

Example prompt for Claude Code — type directly in the conversation:

```
Profile the performance of this scene. Start with a quick CSV profile, then
if there are issues, do a detailed Insights trace to find the cause.
```

What each level returns:

Level	**Data**
CSV Profiler	Per-thread timing (avg/p50/p95/p99), draw calls, memory, FPS, budget violations, trend (stable/degrading/improving)
Insights: analyze_trace	Function name, source file/line, call count, total/avg/max inclusive time per scope
Insights: get_trace_spikes	Frames exceeding budget with the top contributing scopes per frame
Insights: get_trace_frame_summary	Frame count, avg/min/max frame time, FPS, p50/p90/p95/p99 percentiles
Insights: get_trace_net_summary	Per-connection packet/byte totals (in/out + drop rate), top objects by outbound replication bandwidth, top event types (RPCs / replicated properties) by bandwidth. Requires a multiplayer trace — pair with

Level	Data
	`start_pie("listen_server", N)` for automated capture.

Tips for Best Results

- **Be specific about asset paths.** Tell the agent where to save the asset (e.g., `/Game/VFX/NS_MyEffect`) so it doesn't have to guess.
- **Describe the visual result.** "Long thin rain streaks falling at an angle" gives better results than "make rain."
- **Iterate.** Ask the agent to create the effect, preview it in the editor, then ask for adjustments ("make the particles bigger", "add more gravity", "change the color to blue").
- **Close the asset editor** before asking the agent to modify an existing Niagara system or Blueprint. The editor caches its own copy and won't reflect external changes made via Python.
- **For performance analysis,** capture traces during representative workloads — PIE play sessions, complex scenes, or stress tests yield more useful data than idle editor traces.